

AFRILANG Stdlib

std/m/wordtok

Bibliothèque AFRILANG `std/m/wordtok` (niveau medium, catégorie medium). Elle expose 5 fonction(s) : tokenize, tokenCoun

```
`import "std/m/wordtok"` · `use wordtok`
```

- `tokenize(s text) ? list text`
- `tokenCount(s text) ? number`
- `longestToken(s text) ? text`
- `shortestToken(s text) ? text`
- `avgTokenLen(s text) ? number`

Category: medium | Tier: medium | Functions: 5

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/m/wordtok` (niveau medium, catégorie medium). Elle expose 5 fonction(s) : tokenize, tokenCoun

```
`import "std/m/wordtok"` · `use wordtok`  
- `tokenize(s text) ? list text`  
- `tokenCount(s text) ? number`  
- `longestToken(s text) ? text`  
- `shortestToken(s text) ? text`  
- `avgTokenLen(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/wordtok"  
use wordtok
```

Niveau : medium · Catégorie : Modules medium (m/)
Bibliothèques intermédiaires ? import std/m/?

3. Référence API

```
? tokenize(s text) ? list  
? tokenCount(s text) ? number  
? longestToken(s text) ? text  
? shortestToken(s text) ? text  
? avgTokenLen(s text) ? number
```

4. Exemples d'utilisation

```
import "std/m/wordtok"  
use wordtok  
  
create result = tokenize(/* args */)   
say result  
  
create result = tokenCount(/* args */)   
say result  
  
create result = longestToken(/* args */)   
say result  
  
create result = shortestToken(/* args */)   
say result  
  
create result = avgTokenLen(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/m/wordtok"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module wordtok  
  
function tokenize(s text) returns list text  
end  
  
function tokenCount(s text) returns number  
end  
  
function longestToken(s text) returns text
```

end

function shortestToken(s text) returns text

end

function avgTokenLen(s text) returns number

end

end

ENGLISH

1. Overview

AFRILANG library `std/m/wordtok` (tier medium, category medium). Exports 5 function(s): tokenize, tokenCount, longestTok

```
`import "std/m/wordtok"` · `use wordtok`  
- `tokenize(s text) ? list text`  
- `tokenCount(s text) ? number`  
- `longestToken(s text) ? text`  
- `shortestToken(s text) ? text`  
- `avgTokenLen(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/wordtok"  
use wordtok
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries ? import std/m/?

3. API reference

```
? tokenize(s text) ? list  
? tokenCount(s text) ? number  
? longestToken(s text) ? text  
? shortestToken(s text) ? text  
? avgTokenLen(s text) ? number
```

4. Usage examples

```
import "std/m/wordtok"  
use wordtok  
  
create result = tokenize(/* args */)   
say result  
  
create result = tokenCount(/* args */)   
say result  
  
create result = longestToken(/* args */)   
say result  
  
create result = shortestToken(/* args */)   
say result  
  
create result = avgTokenLen(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/m/wordtok"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module wordtok  
  
function tokenize(s text) returns list text  
end  
  
function tokenCount(s text) returns number  
end  
  
function longestToken(s text) returns text
```

end

function shortestToken(s text) returns text

end

function avgTokenLen(s text) returns number

end

end